

I: Impossible Primebox

Source file name: `impossible.c`, `impossible.cpp`, `impossible.java`, or `impossible.py`

Author: Julián Badillo

“I got a challenge for you”, says Danny O’sea - famous career swindler and heistwoman, “I need you to open this very secure safe, called the *Primebox*. I promise you, the pay is worth it.”

The rumor around town is that Danny O’sea is running the most awesome heist of her time: To steal the Ultra Luxurious McGuffin, currently displayed in the Extremely Securest Room of the Very Secure Gallery, and on top of that, locked inside the *Primebox*.

“I have acquired, at a great cost, crucial information about how the *Primebox* works”, she tells you.

The *Primebox* is composed of L *primelocks*, numbered $0, 1, \dots, L-1$. The primelock i is a piece of programming code of the form:

```
prlck i:
x = x[ * Ai ][ + Bi ]
if Pi div x jumpto {prlck ji | end} else jumpto {prlck mi | end}
```

where A_i and B_i are positive integers, and P_i is a prime number ($0 \leq i, j_i, m_i < L$). The expressions in square brackets are optional. For expressions in curly brackets separated by the character “|”, exactly one of them is to be chosen. The notation $s \text{ div } t$ means “ s divides t ”.

The primelock i has an *activation number* $K_i \geq 0$, whose meaning will become clear next. The locks are arranged sequentially and are activated (i.e., executed) starting from primelock 0 and an initial integer value of $x \geq 0$. The execution of a primelock is accomplished in a ‘natural way’ (please, do not ask what ‘natural’ means ...) and the whole process ends when any of the `jumpto end` instructions are executed.

To open the whole *Primebox*, the i -th primelock must be *activated* exactly K_i times. There might be several initial numbers that may activate the primelocks according to their corresponding activation numbers. The *key* to the *Primebox* is the minimum of them.

“Ah! Don’t worry about it, I got one more piece of intelligence”, says Danny O’Sea, “The key does exist and it’s lower than 50 000”.

Your job is, given the primelocks’ source code and their respective activation numbers, to crack the key. A juicy reward is waiting for you.

For instance, consider a *Primebox* defined with the following primelocks:

```
prlck 0:
x = x * 2 + 3
if 3 div x jumpto prlck 2 else jumpto prlck 1
prlck 1:
x = x * 4 + 1
if 5 div x jumpto prlck 0 else jumpto prlck 2
prlck 2:
x = x + 5
if 7 div x jumpto prlck 1 else jumpto end
```

and activation numbers $K_0 = 1$, $K_1 = 2$, and $K_2 = 2$. You may check that, starting with $x = 10$, the process will end after the primelocks are activated, as requested. However, this is not the case with any integer less than 10. Therefore, this *Primebox* is opened with the key 10.

Input

The input consists of several test cases. Notice that all variables, integers, and operators described here are separated by single blanks. Each test case defines a Primebox and starts with an integer L ($1 \leq L \leq 10$), the number of primelocks. The following $3 \times L$ lines describe the primelocks in sequential order, each one following the format described above. You may suppose that, for primelock i ($0 \leq i < L$), the involved parameters A_i , B_i , and P_i are positive integers ($0 \leq A_i, B_i, P_i \leq 1\,000$), P_i is a prime number, and every referenced primelock number is in the integer interval $0 \dots L - 1$. The case ends with a line containing L numbers K_i ($0 \leq i < L$ and $0 \leq K_i \leq 1\,000$), the corresponding activation numbers for the given primelocks. The input ends with the line 0, which should not be processed.

The input must be read from standard input.

Output

For each test case, output a single line with the corresponding cracked key.

The output must be written to standard output.

Sample Input	Sample Output
<pre> 3 prlck 0: x = x * 2 + 3 if 3 div x jumpto prlck 2 else jumpto prlck 1 prlck 1: x = x * 4 + 1 if 5 div x jumpto prlck 0 else jumpto prlck 2 prlck 2: x = x + 5 if 7 div x jumpto prlck 1 else jumpto end 1 2 2 3 prlck 0: x = x + 3 if 71 div x jumpto end else jumpto prlck 1 prlck 1: x = x * 15 if 131 div x jumpto end else jumpto prlck 2 prlck 2: x = x * 24 + 1 if 3 div x jumpto prlck 1 else jumpto prlck 0 7 7 6 0 </pre>	<pre> 10 255 </pre>